

BernatLab

Operativa 24/7, monitoratge i manteniment

Raspberry Pi · Docker · IA · LoRa · Hort Osona

Mòdul 6

Bernat — 8 de juliol del 2026

Document tècnic de consulta personal

Sobre aquest manual

Aquest és el segon mòdul del manual tècnic del BernatLab. Cobreix tota la cadena de sensors i visualització: el protocol MQTT, el broker Mosquitto, l'esquema de publicació dels sensors, la base de dades InfluxDB, l'agent Telegraf, la programació visual amb Node-RED, la visualització amb Grafana, l'API REST amb FastAPI, la integració amb la web pública Hort Osona i l'operativa del dia a dia.

Com es llegeix

En ordre, si parteixes de zero. Per capítols, si ja tens una base i vols aprofundir. Cada capítol segueix la mateixa estructura: teoria, aplicació al BernatLab, esquemes, comandes útils, errors habituals, exercicis pràctics i resum final.

Índex de capítols

- 11. Del Mòdul 1 al M2: què construïm
- 12. MQTT des de zero
- 13. Mosquitto al BernatLab
- 14. Publicar dades: els sensors
- 15. InfluxDB: base de dades de sèries temporals
- 16. Telegraf: el pont
- 17. Node-RED: programació visual
- 18. Fluxos pràctics
- 19. Grafana: visualitzar les dades
- 20. API pública: servir les dades al món
- 21. Integració amb Hort Osona
- 22. Operativa: còpies, alertes, escalat

Capítol 51 — Filosofia operativa: del DIY al servei 24/7

"Un servidor personal no és un experiment: és un servei. I com a tal, cal tractar-lo."

51.1 El salt mental: de provar a operar

Quan comences amb un homelab, estàs en mode **experiment**: proves coses, toques, trenques, repares. Però en algun moment, el servidor comença a allotjar coses que **importen**:

- Dades de sensors.
- Una web pública.
- Un assistent d'IA.
- Còpies de seguretat.
- Un bot de Telegram.

Llavors cal canviar la mentalitat: ja no estàs experimentant, estàs **operant un servei**. I això implica:

- **Fiabilitat**: el servei ha d'estar disponible.
- **Recuperabilitat**: si falla, ha de poder tornar-se a aixecar.
- **Observabilitat**: has de saber què passa.
- **Mantenibilitat**: el sistema ha de ser fàcil d'actualitzar.

51.2 SLA personal

Un **SLA** (Service Level Agreement) és un contracte sobre la qualitat d'un servei. En un homelab, ets **l'empresa i el client alhora**. Defineix el teu SLA personal:

- **Disponibilitat objectiu**: 99% (3.6 dies de caiguda a l'any) o 99.9% (8.8 hores a l'any)?
- **Temps de resposta**: 24 h per a incidents crítics.
- **Temps de resolució**: 4-8 h per a incidents crítics.
- **Manteniment programat**: dilluns de 3 a 5 de la matinada?

Això és flexible, però cal posar-ho per escrit.

51.3 El principi de "boring tech"

Boring tech (tecnologia avorrida) és tecnologia que:

- Funciona sense sorpreses.
- Té anys de maduresa.
- Té bona documentació.
- Té comunitat gran.

- No és la més nova ni la més brillant.

Per al BernatLab, "boring tech" seria:

- **Debian** (no Arch, no NixOS).
- **Docker** (no Kubernetes, no Podman).
- **Docker Compose** (no Helm, no K8s manifests).
- **restic** (no fer la teva pròpia eina de còpies).
- **Prometheus + Grafana** (no construir el teu propi sistema de mètriques).
- **Tailscale** (no muntar la teva pròpia VPN).

Això redueix el temps de manteniment i la probabilitat de problemes.

51.4 El cicle operatiu

Un cop tens el sistema en producció, hi ha un cicle operatiu:

1. **Monitorar**: veure què passa.
2. **Alertar**: avisar quan alguna cosa va malament.
3. **Diagnosticar**: trobar la causa.
4. **Resoldre**: arreglar el problema.
5. **Documentar**: registrar què ha passat.
6. **Aprendre**: millorar per evitar que es repeteixi.

Això és un bucle continu. Com més madur és el sistema, més ràpid és el cicle.

51.5 Què cal monitorar

Al BernatLab, monitorem:

- **Disponibilitat**: serveis funcionant o caiguts.
- **Rendiment**: CPU, RAM, disc, xarxa.
- **Salut del sistema**: temperatura, errors de disc, etc.
- **Tràfic de xarxa**: ample de banda utilitzat.
- **Certificats**: quan caduquen.
- **Còpies**: quan es fan i quan fallen.
- **Logs de seguretat**: intents d'intrusió.
- **Dades específiques**: lectures de sensors, etc.

51.6 Cicles d'actualització

Defineix una cadència:

- **Diàriament**: comprovar alertes.

- **Setmanalment:** revisar logs, actualitzar sistema.
- **Mensualment:** auditar seguretat, fer neteja.
- **Trimestralment:** revisar configuració, planificar canvis.
- **Anualment:** revisar el DRP, planificar hardware.

Si no tens cadència, les coses es degraden. Els discos s'omplen, els registres es caducen, les actualitzacions s'acumulen.

51.7 El concepte d'"on-call"

En empreses, hi ha un equip d'**on-call** (guàrdia) que respon a incidents 24/7. En un homelab, **tu ets l'on-call**. Però pots ser-ho de manera sostenible:

1. **Defineix horaris:** no vols rebre alertes a les 3 de la matinada tots els dies.
2. **Diferencia per severitat:** alertes crítiques (24/7) i no crítiques (horari laboral).
3. **Automatitza el que puguis:** les alertes que no necessiten acció humana s'han de resoldre soles.
4. **Tingues un pla B:** si no pots respondre, qui més pot?

51.8 Mantenibilitat

Un sistema mantenible és:

- **Documentat:** cada component té una descripció.
- **Reproducible:** pots muntar-lo de zero amb les instruccions.
- **Modular:** cada component es pot canviar sense trencar la resta.
- **Testejat:** tens proves que validen que funciona.

Per al BernatLab, això es tradueix en:

- Un **README complet** amb totes les passes d'instal·lació.
- Un **docker-compose.yml** que defineix tots els serveis.
- **Scripts d'instal·lació** idempotents (es poden executar múltiples cops).
- **Tests** que validen els components crítics.

51.9 El temps com a factor

Un dels factors que la gent no considera és el **temps**. Cada decisió tècnica té un cost de temps:

- Triar Kubernetes: 100+ hores d'aprenentatge.
- Triar Docker Compose: 5-10 hores d'aprenentatge.
- Triar una eina de mètriques nova: 20-50 h.
- Aprendre una nova tecnologia: variable.

Quan planegis, pregunta't: **val la pena el temps invertit?** Si tens 10 hores, és millor:

- 5 hores a configurar alerting → beneficis immediats.
- 5 hores a provar una tecnologia nova → beneficis dubtosos.

51.10 El factor "embarrassment-driven development"

A vegades, la millor motivació per fer les coses bé és **la vergonya**:

- Tens una gràfica de pèrdua de dades? No voldràs ensenyar-la.
- Tens una web que es cau cada 2 dies? No la voldràs compartir.
- Tens un sistema que triga 3 dies a recuperar? No voldràs explicar-ho.

Fes servir la vergonya constructiva. Documenta les coses que **no** vols que es repeteixin.

51.11 Filosofia del "menys és més"

Al BernatLab, **menys serveis** vol dir:

- Menys coses que poden fallar.
- Menys temps de manteniment.
- Menys dependències.
- Menys complexitat.

Cada nou servei ha de **justificar el seu cost de temps**. Si no t'aporta valor clar, no l'afegeixis.

51.12 Cultura de la documentació

Un bon sistema operatiu té bona documentació. La documentació:

- **Permet que qualsevol** (inclòs tu del futur) entengui el sistema.
- **Redueix errors** en canvis.
- **Facilita la recuperació** quan falla.
- **Comunica coneixement** a altres persones (família, amics).

Al BernatLab, documenta:

- L'arquitectura (README).
- Els procediments operatius (DRP, runbooks).
- Els canvis (CHANGELOG).
- Les decisions (ADR - Architecture Decision Records, registres de decisions arquitectòniques).

51.13 Indicadors que vas bé

Senyals que el sistema és madur:

- **Pocs incidents** al mes.

- **Recuperació ràpida** quan passa alguna cosa.
- **Actualitzacions regulars** sense trencar res.
- **Mètriques bones** (latència baixa, disponibilitat alta).
- **Documentació actualitzada**.

Si tot això es compleix, el sistema està sa.

51.14 Quan replantejar-se

De vegades, cal fer canvis radicals:

- El sistema creix massa per la Raspberry.
- Una tecnologia queda obsoleta.
- Les necessitats canvien (per exemple, l'hort creix i cal més capacitat).
- Apareix una millor alternativa.

Replantejar-se cada 2-3 anys és sa. No tinguis por de migrar si la nova opció és clarament millor.

51.15 Resum

L'operativa 24/7 és una mentalitat, no un producte. Boring tech, cicles regulars, observabilitat, mantenibilitat, documentació. Al proper capítol veurem el monitoratge avançat amb Grafana i Prometheus.

51.16 Exercicis pràctics

1. Defineix el teu SLA personal.
2. Inventaria tots els serveis i classifica'ls per criticitat.
3. Defineix els cicles d'actualització (diari, setmanal, mensual).
4. Crea una checklist de manteniment setmanal.
5. Documenta l'arquitectura actual al README.
6. Escribeu el primer ADR (per què has triat Docker, per exemple).
7. Comença un CHANGELOG.md.

Capítol 52 — Monitoratge avançat amb Grafana i Prometheus

“Si no pots mesurar-ho, no pots millorar-ho. Si no pots veure-ho, no saps què falla.”

52.1 Què és el monitoratge

Monitorar un sistema significa **recollir mètriques contínuament** per entendre què passa. Al BernatLab, això es fa amb tres eines clau:

- **Prometheus**: recull mètriques (números) dels serveis.
- **Grafana**: visualitza les mètriques en gràfiques i panells.
- **Alertmanager**: gestiona les alertes basades en les mètriques.

Aquesta combinació s'anomena **stack PGE** (Prometheus, Grafana, Exporters).

52.2 Com funciona Prometheus

Prometheus funciona amb un model **pull** (empènyer vs estirar):

1. Prometheus **demana** mètriques a cada servei cada pocs segons.
2. Els serveis exposen les mètriques a `/metrics` en format text.
3. Prometheus les emmagatzema a la seva base de dades.
4. Grafana les consulta i les dibuixa.

Exemple de mètriques d'un contenidor:

```
# HELP container_cpu_usage_seconds_total CPU usat pel contenidor
# TYPE container_cpu_usage_seconds_total counter
container_cpu_usage_seconds_total{name="grafana"} 12.5

# HELP container_memory_usage_bytes Memòria usada pel contenidor
# TYPE container_memory_usage_bytes gauge
container_memory_usage_bytes{name="grafana"} 52428800
```

52.3 Instal·lació de Prometheus

A la Raspberry amb Docker Compose

Crea `~/homelab/compose/monitoring.yml`:

```
version: "3.8"

services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: unless-stopped
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
      - ./prometheus/data:/prometheus
    command:
```



```

- '--config.file=/etc/prometheus/prometheus.yml'
- '--storage.tsdb.path=/prometheus'
- '--web.console.libraries=/usr/share/prometheus/console_libraries'
- '--web.console.templates=/usr/share/prometheus/consoles'
ports:
- "9090:9090"
networks:
- monitoring

```

```

grafana:
  image: grafana/grafana:latest
  container_name: grafana
  restart: unless-stopped
  depends_on:
    - prometheus
  volumes:
    - ./grafana/data:/var/lib/grafana
    - ./grafana/provisioning:/etc/grafana/provisioning
  environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD}
    - GF_USERS_ALLOW_SIGN_UP=false
  ports:
    - "3000:3000"
  networks:
    - monitoring

```

```

networks:
  monitoring:
    driver: bridge

```

Crea els volums:

```

mkdir -p ~/homelab/compose/prometheus/data
mkdir -p ~/homelab/compose/grafana/data
mkdir -p ~/homelab/compose/grafana/provisioning/datasources
mkdir -p ~/homelab/compose/grafana/provisioning/dashboards

```

52.4 Configuració de Prometheus

Crea ~/homelab/compose/prometheus/prometheus.yml:

```

global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    monitor: bernatlab

scrape_configs:
  # El propi Prometheus
  - job_name: prometheus
    static_configs:
      - targets: ['localhost:9090']

  # Node Exporter (mètriques de la Raspberry)
  - job_name: node
    static_configs:
      - targets: ['node-exporter:9100']

  # cAdvisor (mètriques de Docker)
  - job_name: cadvisor
    static_configs:
      - targets: ['cadvisor:8080']

```

```
# Altres serveis HTTP
- job_name: bernatlab-services
  metrics_path: /metrics
  static_configs:
    - targets:
      - 'mosquitto:9001'
      - 'influxdb:8086'
      - 'uptime-kuma:3001'
```

Afegeix **node-exporter** i **cadvisor** al compose:

```
node-exporter:
  image: prom/node-exporter:latest
  container_name: node-exporter
  restart: unless-stopped
  pid: host
  volumes:
    - /proc:/host/proc:ro
    - /sys:/host/sys:ro
    - /:/rootfs:ro
  command:
    - '--path.procfs=/host/proc'
    - '--path.sysfs=/host/sys'
    - '--path.rootfs=/rootfs'
    - '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)($|/)'
  networks:
    - monitoring

cadvisor:
  image: gcr.io/cadvisor/cadvisor:latest
  container_name: cadvisor
  restart: unless-stopped
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
    - /dev/disk/:/dev/disk:ro
  privileged: true
  networks:
    - monitoring
```

Engega-ho tot:

```
cd ~/homelab/compose
docker compose -f monitoring.yml up -d
```

52.5 Com accedir-hi

- **Prometheus:** <http://100.115.134.76:9090>
- **Grafana:** <http://100.115.134.76:3000>

A Grafana, el primer accés et demanarà contrasenya. L'has definit al compose.

52.6 Configurar Grafana

Afegir Prometheus com a font de dades

Crea ~/homelab/compose/grafana/provisioning/datasources/prometheus.yml:

```
apiVersion: 1
```

```
datasources:  
  - name: Prometheus  
    type: prometheus  
    access: proxy  
    url: http://prometheus:9090  
    isDefault: true  
    editable: true
```

Grafana carregarà automàticament aquesta configuració.

Crear un dashboard bàsic

Al panell, fes clic a **+** → **Dashboard** → **Add visualization**.

Exemple de panell "Ús de CPU de la Raspberry":

- **Data source:** Prometheus
- **Metric:** `100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`
- **Visualization:** Time series
- **Title:** CPU Raspberry (%)

Això et mostrarà una gràfica en temps real de l'ús de CPU.

52.7 Mètriques útils per al BernatLab

Sistema (Node Exporter)

- **CPU:** `rate(node_cpu_seconds_total[5m])`
- **RAM:** `node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes`
- **Disc:** `(node_filesystem_size_bytes - node_filesystem_avail_bytes) / node_filesystem_size_bytes`
- **Xarxa:** `rate(node_network_receive_bytes_total[5m])`
- **Temperatura:** `node_thermal_zone_temp` (si tens el sensor)

Docker (cAdvisor)

- **CPU per contenidor:** `rate(container_cpu_usage_seconds_total[5m])`
- **RAM per contenidor:** `container_memory_usage_bytes`
- **Xarxa per contenidor:** `rate(container_network_receive_bytes_total[5m])`

Mosquitto (mètriques natives)

- **Missatges publicats:** `mosquitto_messages_published_total`
- **Missatges rebuts:** `mosquitto_messages_received_total`
- **Connexions actives:** `mosquitto_clients_connected`

52.8 Dashboards prefabricats

Grafana té una comunitat amb milers de dashboards. Per trobar-los:

1. Vés a <https://grafana.com/grafana/dashboards>.
2. Busca "Node Exporter Full" (ID: 1860).
3. Copia l'ID.
4. A Grafana: **+** → **Import** → enganxa l'ID.

Altres dashboards útils:

- **Docker Container Monitoring** (ID: 893): mètriques dels contenidors.
- **Prometheus Stats** (ID: 2): mètriques del propi Prometheus.
- **Mosquitto** (ID: 11587): mètriques MQTT.

52.9 Retenció de dades

Per defecte, Prometheus guarda les dades 15 dies. Per canviar-ho:

```
docker compose -f monitoring.yml down
# Afegeix a la comanda de prometheus:
- '--storage.tsdb.retention.time=30d'
docker compose -f monitoring.yml up -d
```

Això guarda 30 dies. Més dies = més espai en disc.

52.10 Què fer amb les dades

Un cop tens gràfiques, pots:

- **Detectar patrons:** la CPU puja cada dia a les 15h? Probablement una tasca programada.
- **Identificar pics:** la RAM s'ha disparat a les 3 de la matinada? Alguna cosa consumeix molt.
- **Comparar períodes:** aquest mes vs. el mes passat.
- **Predir necessitats:** si el creixement de dades és X%/mes, quan s'omplirà el disc?

52.11 Compartir dashboards

Grafana permet exportar i compartir dashboards:

1. **Exportar JSON:** a la configuració del dashboard, **Share** → **Export** → **Save to file**.
2. **Importar JSON:** **+** → **Import** → puja el fitxer.
3. **Compartir URL:** **Share** → **Link**. Qualsevol amb la URL pot veure (si és a la mateixa xarxa).

52.12 Limitacions de Prometheus

Prometheus és excel·lent, però té limitacions:

- **Només numèrics:** no emmagatzema logs ni traces.
- **No escal·la horitzontalment** (per a moltes dades, cal Cortex, Thanos, o Mimir).
- **No té UI per a alertes avançades** (cal Alertmanager).
- **No és bo per a alta cardinalitat** (moltes etiquetes úniques).

Per al BernatLab, és perfecte. Per a una empresa, caldrien eines més potents.

52.13 Com estendre el monitoratge

Quan vulguis afegir més mètriques:

1. **Afegeix exporters:** serveis que exposen mètriques en format Prometheus.
2. **Usa pushgateway:** per a mètriques de treballs curts (cron, scripts).
3. **Configura alertes:** per a mètriques crítiques.
4. **Integra amb Telegram:** per a notificacions.

52.14 Què NO monitorar

No totes les mètriques aporten valor:

- **Mètriques que no canvien mai:** inútils.
- **Mètriques massa granulars:** generen molt emmagatzematge sense benefici.
- **Mètriques que no pots actuar:** si no hi ha res a fer, no cal saber-ho.

La regla: **si no saps quina acció prendràs quan vegis aquesta mètrica, no la monitoris.**

52.15 Resum

El monitoratge és la base de l'operativa. Prometheus recull, Grafana visualitza, i tu pots prendre decisions. Al BernatLab, amb Node Exporter + cAdvisor + exporters dels serveis tens visibilitat completa. Al proper capítol veurem com configurar alertes per Telegram.

52.16 Exercicis pràctics

1. Desplega Prometheus i Grafana a la Raspberry.
2. Configura Node Exporter i cAdvisor.
3. Importa el dashboard "Node Exporter Full".
4. Crea un panell personalitzat de CPU per contenidor.
5. Afegeix mètriques de Mosquitto.
6. Exporta un dashboard i comparteix-lo.
7. Configura la retenció a 30 dies.
8. Documenta al README els URLs de Prometheus i Grafana.

Capitol 53 — Alertes intel·ligents amb Grafana i Telegram

"Una alerta que no actues és soroll. Una alerta que arriba quan toca, al canal adequat, és or."

53.1 Què és una alerta útil

Una alerta útil té quatre propietats:

1. **Rellevant**: avisa d'un problema real, no d'una molèstia.
2. **Accionable**: saps què fer quan la reps.
3. **Puntual**: arriba quan toca, no més tard ni massa aviat.
4. **Al canal correcte**: crítiques a Telegram immediatament, informatives al correu en horari laboral.

Si una alerta no té aquestes quatre propietats, és soroll.

53.2 Tipus d'alertes

Alertes de disponibilitat

- Un servei està caigut.
- Un contenidor ha deixat de respondre.
- La Raspberry no és accessible per SSH.

Alertes de rendiment

- La CPU porta més de 30 minuts al 90%.
- El disc s'omplirà en menys de 24 h.
- La memòria lliure és inferior al 10%.

Alertes de seguretat

- Més de 10 intents SSH fallits en 5 min.
- Un usuari ha accedit des d'una IP nova.
- Una alerta de fail2ban s'ha activat.

Alertes de dades

- Un node LoRa no ha enviat dades en més d'1 hora.
- La còpia de seguretat no s'ha fet.
- InfluxDB té errors de consulta.

53.3 Alertmanager: el cervell de les alertes

Alertmanager (integrat a Prometheus) gestiona les alertes:

1. **Rep** les alertes de Prometheus.
2. **Agrupar** alertes relacionades.
3. **Silencia** alertes que saps que estan passant.
4. **Enruta** a canals diferents segons la severitat.
5. **Inhibeix** alertes que són conseqüència d'altres.

Configuració bàsica a `~/home/lab/compose/prometheus/alertmanager.yml`:

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname', 'instance']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: 'telegram-critiques'
  routes:
    - match:
        severity: critical
      receiver: 'telegram-critiques'
    - match:
        severity: warning
      receiver: 'telegram-avís'

receivers:
  - name: 'telegram-critiques'
    webhook_configs:
      - url: 'http://alertmanager-webhook:8080/alerts/critical'

  - name: 'telegram-avís'
    webhook_configs:
      - url: 'http://alertmanager-webhook:8080/alerts/warning'

inhibit_rules:
  - source_match:
      severity: critical
    target_match:
      severity: warning
    equal: ['alertname', 'instance']
```

Afegeix Alertmanager al compose:

```
alertmanager:
  image: prom/alertmanager:latest
  container_name: alertmanager
  restart: unless-stopped
  volumes:
    - ./prometheus/alertmanager.yml:/etc/alertmanager/alertmanager.yml
  command:
    - '--config.file=/etc/alertmanager/alertmanager.yml'
  ports:
    - "9093:9093"
  networks:
    - monitoring
```

53.4 Regles d'alerta a Prometheus

Crea ~/homelab/compose/prometheus/rules.yml:

```
groups:
- name: bernatlab
  interval: 30s
  rules:
    # CPU alta durant 5 minuts
    - alert: HighCPUUsage
      expr: 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 90
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "CPU alta a {{ $labels.instance }}"
        description: "CPU al {{ $value }}% durant més de 5 minuts."

    # RAM baixa
    - alert: LowMemory
      expr: (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100 < 10
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "RAM baixa a {{ $labels.instance }}"
        description: "Només queda {{ $value }}% de RAM lliure."

    # Disc gairebé ple
    - alert: DiskFull
      expr: (node_filesystem_avail_bytes{fstype!="tmpfs"} / node_filesystem_size_bytes) * 100 < 10
      for: 10m
      labels:
        severity: critical
      annotations:
        summary: "Disc ple a {{ $labels.instance }}"
        description: "Només queda {{ $value }}% d'espai lliure."

    # Servei caigut (ping)
    - alert: ServiceDown
      expr: up{job="bernatlab-services"} == 0
      for: 2m
      labels:
        severity: critical
      annotations:
        summary: "Servei {{ $labels.instance }} caigut"
        description: "El servei no respon a ping des de fa 2 minuts."

    # Temperatura alta
    - alert: HighTemperature
      expr: node_thermal_zone_temp > 75
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Temperatura alta: {{ $value }}°C"
        description: "La Raspberry està a {{ $value }}°C, considera netejar-la o millorar la ventilació"
```

Carrega les regles a `prometheus.yml`:

```
rule_files:
- "rules.yml"
```

alerting:


```

alertmanagers:
  - static_configs:
    - targets: ['alertmanager:9093']

```

53.5 Crear el bot de Telegram

Per enviar alertes a Telegram:

1. Obre Telegram i parla amb **@BotFather**.
2. Envia `/newbot` i segueix les instruccions.
3. Guarda el **token** que et dóna (exemple: `1234567890:ABCdefGHI...`).
4. Crea un grup on el bot i tu sigueu membres.
5. Afegeix el bot al grup.
6. Obtenir el **chat_id** del grup: visita `https://api.telegram.org/bot<TOKEN>/getUpdates` i mira el `chat.id`.

53.6 Servei de webhook a Telegram

Crea un petit servei que rebí alertes d'Alertmanager i les envii a Telegram. Crea `~/homelab/compose/alert-webhook/`:

app.py:

```

import os
import json
import requests
from flask import Flask, request

app = Flask(__name__)

TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN")
TELEGRAM_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID")

def send_telegram(message):
    url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
    requests.post(url, json={
        "chat_id": TELEGRAM_CHAT_ID,
        "text": message,
        "parse_mode": "HTML"
    })

@app.route("/alerts/critical", methods=["POST"])
def critical():
    data = request.json
    for alert in data.get("alerts", []):
        status = alert.get("status", "unknown")
        if status == "resolved":
            emoji = "■"
            text = "RESOLT"
        else:
            emoji = "■"
            text = "ALERTA CRÍTICA"
        summary = alert.get("annotations", {}).get("summary", "Sense resum")
        description = alert.get("annotations", {}).get("description", "")
        message = f"{emoji} <b>{text}</b>\n\n{summary}\n{description}"
        send_telegram(message)

```

```

        return "OK", 200

@app.route("/alerts/warning", methods=["POST"])
def warning():
    data = request.json
    for alert in data.get("alerts", []):
        status = alert.get("status", "unknown")
        if status == "resolved":
            emoji = "■"
            text = "RESOLT"
        else:
            emoji = "■■■"
            text = "AVÍS"
        summary = alert.get("annotations", {}).get("summary", "")
        message = f"{emoji} <b>{text}</b>\n\n{summary}"
        send_telegram(message)
    return "OK", 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)

```

Dockerfile:

```

FROM python:3.11-slim
WORKDIR /app
RUN pip install flask requests gunicorn
COPY app.py .
CMD ["gunicorn", "-b", "0.0.0.0:8080", "-w", "2", "app:app"]

```

requirements.txt:

```

flask==3.0.0
requests==2.31.0
gunicorn==21.2.0

```

Crea .env (afegit a .gitignore):

```

TELEGRAM_TOKEN=1234567890:ABCdefGHIjklMNOpqrSTUVwxyz
TELEGRAM_CHAT_ID=-1001234567890

```

I actualitza el compose:

```

alertmanager-webhook:
  build: ./alert-webhook
  container_name: alertmanager-webhook
  restart: unless-stopped
  env_file:
    - ./alert-webhook/.env
  networks:
    - monitoring

```

53.7 Provar les alertes

Per forçar una alerta de prova:

1. Apaga un contenidor temporalment:

```
docker stop grafana
```

1. Espera 2-3 minuts.

1. Hauries de rebre una alerta a Telegram: "Servei caigut: grafana".

1. Torna a engegar:

```
docker start grafana
```

1. Rebràs una alerta "RESOLT".

53.8 Alertes intel·ligents

Algunes alertes són molt específiques i útils:

Alerta: node LoRa no ha enviat dades en 1h

```
- alert: LoRANodeOffline
  expr: time() - max(temperature_last_seen{node="hort1"}) > 3600
  for: 10m
  labels:
    severity: warning
  annotations:
    summary: "Node LoRa hort1 sense dades des de fa 1h"
    description: "Comprova la bateria o la cobertura."
```

Alerta: còpia de seguretat no s'ha fet

Pots usar Pushgateway per exposar l'edat de l'última còpia:

```
# Script que actualitza la mètrica
AGE=$(stat -c %Y /var/lib/bernatlab-backups/latest)
NOW=$(date +%s)
HOURS=$(( (NOW - AGE) / 3600 ))
echo "bernatlab_backup_age_hours $HOURS" | curl --data-binary @- \
  http://localhost:9091/metrics/job/backup
```

I la regla:

```
- alert: BackupStale
  expr: bernatlab_backup_age_hours > 26
  for: 1h
  labels:
    severity: warning
  annotations:
    summary: "Còpia antiga: {{ $value }} hores"
```

Alerta: certificat a punt de caducar

```
- alert: CertificateExpiring
  expr: (ssl_cert_not_after - time()) / 86400 < 14
  for: 1h
  labels:
    severity: warning
  annotations:
    summary: "Certificat expira en menys de 14 dies"
```

Cal un exporter com **blackbox_exporter** o **ssl_exporter**.

53.9 Silenciació temporal

Quan saps que una cosa pot fallar (manteniment programat, canvis), pots silenciar alertes:

```
# Via amtool
amtool silence add --alertmanager=http://localhost:9093 \
  --comment="Manteniment programat" \
  --duration=2h \
  --match-label=alertname=HighCPUUsage
```

O via interfície web d'Alertmanager: <http://localhost:9093>.

53.10 Evitar el soroll

Si reps masses alertes, ajusta:

- **for**: augmenta el temps (per exemple, de 5m a 15m).
- **threshold**: ajusta el llindar (per exemple, de 90% a 95%).
- **group_by**: agrupa alertes relacionades.
- **repeat_interval**: augmenta l'interval de repetició.

53.11 Errors habituals

Error 1: alertes que ningú llegeix.

Si tens 50 alertes al dia, deixaràs de mirar-les. Sigues estricte amb què mereix una alerta.

Error 2: alertes que es disparen massa.

Si una alerta es dispara cada hora, no és útil. Ajusta el llindar.

Error 3: no provar les alertes.

Si no has provat mai l'alerta, no saps si funciona. Prova-les periòdicament.

Error 4: no documentar què fer.

Cada alerta hauria d'incloure què fer. Un runbook enllaçat a l'anotació.

53.12 Bones pràctiques

1. **Meningi's per cada alerta**: quina acció prendré?
2. **Severitat clara**: critical, warning, info.
3. **Anotacions útils**: resum + descripció + enllaç al runbook.
4. **Provar-les periòdicament**: força una alerta cada mes.
5. **Ajustar el soroll**: si una alerta no és útil, ajusta-la o elimina-la.
6. **Tenir un canal de "tot OK"**: rebre resolucions és igual d'útil que rebre alertes.

53.13 Resum

Les alertes són la part més important del monitoratge: t'avis quan alguna cosa va malament.

Alertmanager + Telegram és una combinació potent i gratuïta. La clau és evitar el soroll i mantenir alertes accionables. Al proper capítol veurem els scripts de manteniment i actualitzacions.

53.14 Exercicis pràctics

1. Crea un bot de Telegram i obté el token.
2. Desplega Alertmanager i el servei de webhook.

3. Configura 5 regles d'alerta a Prometheus.
4. Prova les alertes aturant un servei.
5. Afegeix una alerta específica del teu cas (sensor, còpia, certificat).
6. Configura un silenci per a manteniment.
7. Documenta al README les alertes configurades.

Capítol 54 — Scripts de manteniment i actualitzacions

"Si ho has de fer més de dues vegades, automatitza-ho. Si ho has de fer cada setmana, programa-ho."

54.1 Què automatitzar

Al BernatLab, les tasques repetitives que cal automatitzar són:

- **Actualitzacions del sistema:** apt update + apt upgrade.
- **Actualitzacions de contenidors:** docker compose pull + up.
- **Còpies de seguretat:** restic backup.
- **Neteja:** esborrar logs antics, imatges obsoletes.
- **Rotació de logs:** logrotate.
- **Monitoratge:** alertes, comprovacions periòdiques.
- **Certificats:** renovació automàtica.

54.2 Estructura de scripts

Crea una estructura clara:

```
~/homelab/scripts/
■■■ update-system.sh          # Actualitza el SO
■■■ update-containers.sh      # Actualitza contenidors
■■■ backup-daily.sh           # Còpia diària
■■■ cleanup.sh                 # Neteja
■■■ health-check.sh           # Comprovació de salut
■■■ restart-failed.sh         # Reiniciar contenidors caiguts
■■■ cert-renew.sh             # Renovació de certificats
■■■ lib/
    ■■■ colors.sh             # Codis de color
    ■■■ notify.sh             # Funcions de notificació
```

Tots els scripts han de ser:

- **Idempotents:** es poden executar múltiples vegades sense efectes secundaris.
- **Amb logging:** registren què fan.
- **Amb notificació:** avisen si falla.
- **Amb cleanup:** netegen recursos temporals.

54.3 Plantilla de script

```
#!/bin/bash
#
# update-system.sh - Actualitza el sistema operatiu
#
# Ús: ./update-system.sh [--auto]
#
# Opcions:
#   --auto      No demana confirmació

set -euo pipefail

# Colors
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m'

# Funcions
log() { echo -e "${GREEN}[$(date +%Y-%m-%d %H:%M:%S)]${NC} $*"; }
warn() { echo -e "${YELLOW}[WARN]${NC} $*"; }
err() { echo -e "${RED}[ERROR]${NC} $*" >&2; }

# Comprovació de root
if [ "$(id -u)" -ne 0 ]; then
    err "Aquest script necessita privilegis de root"
    exit 1
fi

# Paràmetres
AUTO=false
[ "${1:-}" == "--auto" ] && AUTO=true

# Lògica
log "Actualitzant la llista de paquets..."
apt update

log "Comprovant què s'ha d'actualitzar..."
UPDATES=$(apt list --upgradable 2>/dev/null | wc -l)
if [ "$UPDATES" -le 1 ]; then
    log "No hi ha actualitzacions pendents."
    exit 0
fi

log "$UPDATES paquets per actualitzar."
if [ "$AUTO" = false ]; then
    read -p "Continuar? (s/N) " -n 1 -r
    echo
    [[ ! $REPLY =~ ^[Ss]$ ]] && exit 0
fi

log "Actualitzant paquets..."
apt upgrade -y

log "Esbarrant paquets obsolets..."
apt autoremove -y

log "Fet!"
```

54.4 Script d'actualització de contenidors

update-containers.sh:

```
#!/bin/bash
set -euo pipefail

cd ~/homelab/compose

for compose_file in *.yaml; do
    log "Actualitzant $compose_file"
    docker compose -f "$compose_file" pull
    docker compose -f "$compose_file" up -d
done

log "Esbarrant imatges antigues..."
docker image prune -f
```

54.5 Script de comprovació de salut

health-check.sh:

```
#!/bin/bash
set -euo pipefail

HEALTHY=0
UNHEALTHY=0
SERVICES=(
    "portainer:9443"
    "uptime-kuma:3001"
    "homepage:3000"
    "grafana:3000"
    "prometheus:9090"
)

for service in "${SERVICES[@]}; do
    name="${service%:*}"
    port="${service#*:}"
    if curl -s -o /dev/null -w "%{http_code}" \
        --connect-timeout 3 \
        "http://localhost:$port" | grep -q "^[2-4]"; then
        log "✓ $name"
        ((HEALTHY++)) || true
    else
        err "✗ $name"
        ((UNHEALTHY++)) || true
    fi
done

log "Resultat: $HEALTHY sans, $UNHEALTHY amb problemes"

# Notificar si hi ha problemes
if [ $UNHEALTHY -gt 0 ]; then
    send_telegram "■■ BernatLab: $UNHEALTHY serveis no responen"
fi
```

54.6 Reiniciar contenidors caiguts

restart-failed.sh:

```
#!/bin/bash
set -euo pipefail
```



```
# Trobar contenidors en estat exited
FAILED=$(docker ps -a --filter "status=exited" --format "{{.Names}}")

if [ -z "$FAILED" ]; then
    log "Cap contenidor caigut."
    exit 0
fi

for container in $FAILED; do
    log "Reiniciant $container..."
    docker restart "$container"
done

send_telegram "■ BernatLab: contenidors reiniciats: $FAILED"
```

54.7 Neteja periòdica

`cleanup.sh`:

```
#!/bin/bash
set -euo pipefail

log "Esbarrant imatges dangling..."
docker image prune -f

log "Esbarrant volums no utilitzats..."
docker volume prune -f

log "Esbarrant xarxes no utilitzades..."
docker network prune -f

log "Esbarrant contenidors aturats..."
docker container prune -f

log "Esbarrant logs antics (>30 dies)..."
find /var/log -name "*.gz" -mtime +30 -delete
find /var/log -name "*.log.*" -mtime +30 -delete

log "Esbarrant còpies de seguretat antigues..."
# Si tens un script de neteja de restic, crida'l
~/homelab/scripts/lib/notify.sh "■ BernatLab: neteja setmanal completada"
```

54.8 Renovació de certificats

Si uses Caddy, els certificats es renoven sols. Si uses Let's Encrypt amb certbot:

`cert-renew.sh`:

```
#!/bin/bash
set -euo pipefail

log "Renovant certificats..."
certbot renew --quiet

log "Recarregant nginx/Caddy..."
docker exec caddy caddy reload 2>/dev/null || systemctl reload nginx

log "Certificats renovats."
```

54.9 Ús de cron

Edita `crontab -e` (com a root):

```
# Actualització del sistema, diumenge a les 3 AM
0 3 * * 0 /home/bernat/homelab/scripts/update-system.sh --auto >> /var/log/bernatlab-update.log 2>&1

# Actualització de contenidors, dilluns a les 3 AM
0 3 * * 1 /home/bernat/homelab/scripts/update-containers.sh >> /var/log/bernatlab-update.log 2>&1

# Còpia de seguretat, cada dia a les 2 AM
0 2 * * * /home/bernat/homelab/scripts/backup-daily.sh >> /var/log/bernatlab-backup.log 2>&1

# Health check, cada 5 minuts
*/5 * * * * /home/bernat/homelab/scripts/health-check.sh >> /var/log/bernatlab-health.log 2>&1

# Reiniciar contenidors caiguts, cada hora
0 * * * * /home/bernat/homelab/scripts/restart-failed.sh >> /var/log/bernatlab-restart.log 2>&1

# Neteja, dissabte a les 4 AM
0 4 * * 6 /home/bernat/homelab/scripts/cleanup.sh >> /var/log/bernatlab-cleanup.log 2>&1

# Renovació de certificats, dilluns a les 4 AM
0 4 * * 1 /home/bernat/homelab/scripts/cert-renew.sh >> /var/log/bernatlab-cert.log 2>&1
```

54.10 Ús de systemd timers (alternativa a cron)

Systemd timers són més moderns i tenen més funcionalitat:

****/etc/systemd/system/bernatlab-backup.service**:**

```
[Unit]
Description=Còpia de seguretat BernatLab
Wants=bernatlab-backup.timer

[Service]
Type=oneshot
ExecStart=/home/bernat/homelab/scripts/backup-daily.sh
User=bernat
```

****/etc/systemd/system/bernatlab-backup.timer**:**

```
[Unit]
Description=Programació còpia BernatLab

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target
```

Activar:

```
sudo systemctl daemon-reload
sudo systemctl enable --now bernatlab-backup.timer
```

Avantatges de systemd timers:

- Es poden veure amb `systemctl list-timers`.
- Tenen logging integrat (`journalctl`).

- Dependències entre serveis.
- Més fàcils de testejar.

54.11 Notificacions

Tots els scripts poden enviar missatges a Telegram. Crea `lib/notify.sh`:

```
#!/bin/bash
# notify.sh - Envia missatges a Telegram

# Carregar variables del .env
if [ -f ~/homelab/.env ]; then
    source ~/homelab/.env
fi

: "${TELEGRAM_TOKEN:?Necessites definir TELEGRAM_TOKEN}"
: "${TELEGRAM_CHAT_ID:?Necessites definir TELEGRAM_CHAT_ID}"

send_telegram() {
    local message="$1"
    curl -s -X POST "https://api.telegram.org/bot${TELEGRAM_TOKEN}/sendMessage" \
        -d chat_id="${TELEGRAM_CHAT_ID}" \
        -d text="$message" \
        -d parse_mode=HTML \
        > /dev/null
}
```

Afegeix als altres scripts:

```
source ~/homelab/scripts/lib/notify.sh
send_telegram "■ BernatLab: còpia completada"
```

54.12 Monitoratge de cron

Si vols saber que el cron s'executa:

```
# Health check que comprova els logs
if ! grep -q "$(date '+%Y-%m-%d')" /var/log/bernatlab-backup.log; then
    send_telegram "■■ Còpia no s'ha executat avui"
fi
```

54.13 Lockfile: evitar execucions simultànies

Si un script pot trigar molt, vols evitar que s'executi dues vegades:

```
LOCKFILE=/tmp/bernatlab-backup.lock

if [ -e "$LOCKFILE" ]; then
    err "Ja hi ha una execució en curs"
    exit 1
fi
trap 'rm -f "$LOCKFILE"' EXIT
touch "$LOCKFILE"

# ... la teva lògica aquí ...
```

54.14 Comprovació de resultats

Un bon script retorna un codi de sortida:

- **0**: èxit.
- **1**: error genèric.
- **2**: error d'arguments.
- **3**: error de permisos.

I usa `set -euo pipefail` per fallar ràpid en errors.

54.15 Errors habituals

Error 1: scripts que fallen silenciosament.

Sense `set -e`, els errors es poden perdre. Usa-lo sempre.

Error 2: scripts que deixen l'estat inconsistent.

Si el script falla a meitat, hauria de netejar. Usa `trap`.

Error 3: no provar els scripts abans de programar-los.

Prova'ls manualment abans de posar-los a cron.

Error 4: massa automatització.

No automatitzis coses que passes un cop. Perd més temps configurant que no pas fent-ho.

54.16 Resum

Els scripts de manteniment són la base de l'operativa. Plantilles clares, idempotència, logging, notifikacions, lockfiles. Cron o systemd timers per programar-los. Telegram per rebre alertes. En el proper capítol veurem els runbooks, que són la cara humana d'aquesta automatització.

54.17 Exercicis pràctics

1. Crea l'estructura `~/homelab/scripts/`.
2. Escribeu un script d'actualització del sistema.
3. Programa'l amb cron o systemd timer.
4. Afegeix notifikacions a Telegram.
5. Crea un health check que s'executi cada 5 min.
6. Documenta els scripts al README.
7. Fes una prova amb un script que simuli un error.

Capítol 55 — Runbooks: procediments pas a pas

"Un runbook és la diferència entre resoldre un incident en 5 minuts o en 5 hores."

55.1 Què és un runbook

Un **runbook** és un document amb **passes exactes** per resoldre un problema concret. A diferència d'un manual general, és:

- **Específic**: per a un problema concret.
- **Pas a pas**: cada pas té un número i una acció clara.
- **Testejat**: ha estat provat almenys un cop.
- **Actualitzat**: reflecteix l'estat actual del sistema.

Els runbooks són la documentació operativa per excel·lència.

55.2 Quan usar un runbook

- **Incidents**: "El Grafana no respon, què faig?"
- **Manteniment**: "Com canvio la còpia de B2 a Wasabi?"
- **Recuperació**: "Com recupero una còpia concreta?"
- **Desplegament**: "Com poso en marxa el node LoRa nou?"
- **Auditoria**: "Com listo tots els usuaris actius?"

55.3 Estructura d'un runbook

Runbook: [Títol del problema]

Síntomes

- Què veus quan passa el problema.
- Com et n'adones que passa.

Diagnòstic

- Passes per confirmar que és aquest problema.
- Comandes concretes a executar.

Solució

- Passes per resoldre.
- Cada pas amb un número.

Verificació

- Com saber que s'ha resolt.
- Què fer si no s'ha resolt.

Prevenció

- Com evitar que passi de nou.
- Canvis a fer al sistema.

Contactes

- Qui avisar.
- Enllaços útils.

Última actualització

- Data i per què.

55.4 Exemple 1: runbook "El Grafana no respon"

Runbook: Grafana no respon

Síntomes

- El navegador mostra "connexió refusada" a `http://localhost:3000`.
- Les gràfiques no es carreguen.
- Altres serveis poden estar bé.

Diagnòstic

1. Comprovar si el contenidor està en marxa:

```
docker ps | grep grafana ``
```

1. Si el contenidor està aturat, mirar els logs: ``bash docker logs grafana --tail 100 ``

1. Si el contenidor està en marxa però no respon, comprovar el port: ``bash ss -tulnp | grep 3000 ``

Solució

Cas A: contenidor aturat

1. Reiniciar: ``bash docker start grafana ``

1. Esperar 30 segons i comprovar: ``bash docker ps | grep grafana curl -s http://localhost:3000/api/health ``

Cas B: contenidor en marxa però no respon

1. Reiniciar: ````bash docker restart grafana ````
1. Si continua sense respondre, mirar els logs per errors: ````bash docker logs grafana --tail 200 ````
1. Si hi ha error de base de dades, restaurar la còpia: ````bash docker stop grafana cp -r ~/backups/grafana/* /var/lib/docker/volumes/grafana-data/_data/ docker start grafana ````

Verificació

- Cridar a `http://localhost:3000` ha de mostrar el login.
- Fer login ha de funcionar.
- Les gràfiques s'han de carregar.

Prevenció

- Comprovar espai en disc (Grafana necessita almenys 1 GB lliure).
- Assegurar-se que el contenidor té memòria suficient (mínim 256 MB).
- Configurar auto-restart (`restart: unless-stopped` al compose).

Contactes

- @bernat a Telegram per a dubtes.

Última actualització

- 2026-07-09 per Bernat: afegit cas d'error de base de dades.

55.5 Exemple 2: runbook "Recuperar una còpia de seguretat"

Runbook: Recuperar una còpia amb restic

Síntomes

- Has esborrat un fitxer per error.
- La Raspberry ha perdut dades.
- Vols comparar amb una versió anterior.

Diagnòstic

1. Comprovar quines còpies tens: ````bash restic -r ~/backups/bernatlab snapshots ````
1. Trobar la còpia amb el fitxer que vols: ````bash restic -r ~/backups/bernatlab find fitxer.txt ````

Solució

1. Restaurar un sol fitxer: ```bash restic -r ~/backups/bernatlab restore latest \ --target /tmp/restored \ --include /path/al/fitxer.txt cp /tmp/restored/path/al/fitxer.txt /path/al/fitxer.txt ```

1. Restaurar tota la còpia més recent: ```bash restic -r ~/backups/bernatlab restore latest \ --target /tmp/restored ```

1. Restaurar una còpia concreta: ```bash restic -r ~/backups/bernatlab restore abc1234 \ --target /tmp/restored ```

1. Restaurar només el directori homelab: ```bash restic -r ~/backups/bernatlab restore latest \ --target /tmp/restored \ --include /home/bernat/homelab ```

Verificació

- Comprovar que els fitxers restaurats existeixen.
- Comparar amb l'original (si encara existeix).

Prevenció

- No esborrar res fins que no hagi comprovat la còpia.
- Fer còpies abans de canvis importants.

55.6 Exemple 3: runbook "Afegir un nou node LoRa"

Runbook: Afegir un nou node LoRa

Síntomes

- Tens un node ESP32 + SX1262 nou que vols afegir a la xarxa.
- El node ja està programat amb el firmware del Mòdul 3.

Diagnòstic

1. Comprovar que el node té el firmware correcte: ```bash # Connectar el node per USB i obrir el monitor sèrie screen /dev/ttyUSB0 115200 ```

1. Verificar que apareix a TTN: - Vés a <https://console.thethingsnetwork.org>. - Applications → bernatlab → Devices. - Hauries de veure el nou node.

Solució

1. Configurar el node a TTN: - Application → bernatlab → End devices → Add end device. - Seleccionar "Enter end device specifics manually". - Posar DevEUI, AppKey, AppEUI. - Triar perfil "LoRaWAN 1.0.4 EU868".

1. Configurar el payload decoder: - Application → Payload formatters → Uplink. - Enganxa el decoder CayenneLPP.

1. Configurar l'integració amb TTN: - Application → Integrations → MQTT. - Activar i apuntar a `mqtt://localhost:1883`.

1. Verificar que arriba al broker MQTT: ```bash mosquitto_sub -h localhost -t "ttn/bernatlab/+up" -v ```

1. Verificar que arriba a InfluxDB: - Vés a Grafana. - Explora → InfluxDB → measurement: `lora_data`. - Hauries de veure el node nou.

Verificació

- El node envia dades cada X minuts.
- Les dades apareixen a Grafana.
- Les alertes funcionen si hi ha valors fora de rang.

Prevenició

- Documentar el node (nom, ubicació, DevEUI).
- Etiquetar físicament el node.
- Fer una còpia del firmware.

55.7 On guardar els runbooks

Crea una carpeta `~/homelab/runbooks/`` amb un fitxer per runbook. La nomenclatura:

- ``RB-001-grafana-down.md``
- ``RB-002-backup-restore.md``
- ``RB-003-add-lora-node.md``

Index al ``README.md``:

Runbooks del BernatLab

Incidents

- [RB-001](#) - Grafana no respon
- [RB-002](#) - Recuperar còpia
- [RB-003](#) - Afegir node LoRa

Manteniment

- [RB-101](#) - Rotar secrets
- [RB-102](#) - Actualitzar contenidors

Recuperació

- [RB-201](#) - Restaurar Raspberry
- [RB-202](#) - Pèrdua de dades

55.8 Com crear un bon runbook

Característiques

1. ****Títol clar****: el problema en 5-10 paraules.
2. ****Síntomes concrets****: què veus exactament.
3. ****Comandes literals****: copy-paste sense errors.
4. ****Branques condicionals****: "si A, fes X. Si B, fes Y."
5. ****Verificació****: com saber que s'ha resolt.
6. ****Prevenició****: com evitar-ho en el futur.

Bones pràctiques

- ****Escriu'l quan resols un incident****, no abans.
- ****Un runbook per incident****, no pas un per categoria.
- ****Actualitza'l**** quan canvis el sistema.
- ****Prova'l**** amb un company o tu mateix.
- ****Inclou captures**** si cal.

55.9 Runbook vs documentació general

Runbook	Documentació general
---	---
Pas a pas	Explicatiu
Per a un problema concret	Per a un tema general
Resolt ja	Pot ser teòric
Testejat	Pot ser aproximat
Llarg (10-20 passes)	Pot ser curt

55.10 Runbooks a GitHub

Al repo del BernatLab, els runbooks són a `homelab/runbooks/`. Això permet:

- ****Versionat****: cada canvi queda registrat.
- ****Col·laboració****: altres poden millorar-los.
- ****Còpia de seguretat****: estan al núvol.
- ****Historial****: pots veure com era abans.

55.11 Plantilla al repo

Crea `homelab/runbooks/TEMPLATE.md`:

Runbook: [Títol]

Síntomes

-

Diagnòstic

- 1.
- 2.

Solució

- 1.
- 2.

Verificació

- []
- []

Prevenció

-

Contactes

-

Última actualització

- Data:
- Per:
- Canvis:

55.12 Com millorar els runbooks

Cada vegada que resols un incident:

1. ****Escriu el runbook**** si no existeix.
2. ****Actualitza'l**** si existeix.
3. ****Prova'l**** la propera vegada.
4. ****Comparteix-lo**** amb la família/amics si pot ajudar.

55.13 Errors habituals

****Error 1: runbooks massa genèrics**.**

"Si tens un problema, mira els logs" no és un runbook. Ha de tenir comandes específiques.

****Error 2: runbooks desactualitzats**.**

Si canvies el sistema, el runbook queda obsolet. Cal revisar-los periòdicament.

****Error 3: runbooks que ningú llegeix**.**

Guarda'ls on es trobin fàcilment. Referencia'ls a les alertes.

****Error 4: runbooks sense prova**.**

Si no l'has provat, pot ser incorrecte. Prova'l!

55.14 Resum

Els runbooks són la documentació operativa per excel·lència. Pas a pas, específics, testejats. Es creen o

55.15 Exercicis pràctics

1. Crea l'estructura `~/homelab/runbooks/`.`
2. Escriu 3 runbooks per a incidents que ja has tingut.
3. Crea un README que indexi tots els runbooks.
4. Prova un runbook amb una situació simulada.
5. Enllaça els runbooks a les alertes de Grafana.
6. Fes commit al repo i comparteix-los.

Capitol 56 — Diagnòstic i troubleshooting 24/7

"El 80% dels problemes es resolen amb 5 comandes. L'objectiu és saber quines 5."

56.1 El mètode general de diagnòstic

Quan alguna cosa falla, segueix aquest mètode:

1. **Identifica el problema:** què falla exactament? Què esperaves que passés?
2. **Recull informació:** què diuen els logs? Quines mètriques? Quins missatges d'error?
3. **Forma una hipòtesi:** quina és la causa més probable?
4. **Prova la hipòtesi:** un canvi segur per verificar.
5. **Si no funciona, torna al pas 2.**
6. **Si funciona, documenta** (runbook) i comparteix.

No saltar passos. No entrar en pànic. Mantenir la calma.

56.2 Les 10 comandes que salven

Quan tot falla, aquestes comandes et donen la informació que necessites:

1. `htop` o `top`

Veure què consumeix CPU/RAM.

```
htop
```

Què buscar:

- Processos consumint molt.
- Molts processos zombi.
- Swap ple.

2. `df -h`

Veure espai en disc.

```
df -h
```

Si una partició està al 100%, és el problema.

3. `du -sh /*`

Trobar què ocupa espai.

```
du -sh /* | sort -h
```

4. free -h

Veure memòria.

```
free -h
```

5. ss -tulnp

Veure quins ports escolten.

```
ss -tulnp
```

6. journalctl -u servei

Logs d'un servei específic.

```
journalctl -u sshd -n 100
```

7. docker ps -a

Estat de tots els contenidors.

```
docker ps -a
```

8. docker logs contenidor

Logs d'un contenidor.

```
docker logs grafana --tail 100
```

9. tail -f /var/log/syslog

Logs del sistema en temps real.

```
tail -f /var/log/syslog
```

10. ip addr o ip route

Configuració de xarxa.

```
ip addr  
ip route
```

56.3 Problemes comuns i solucions

Problema: contenidor reiniciant-se en bucle

Síntoma: `docker ps` mostra el contenidor com a "Restarting".

Diagnòstic:

```
docker logs <contenidor> --tail 50
```

Cerca errors. Sovint és un problema de:

- Configuració incorrecta.
- Volum no muntat.
- Dependència no disponible.

Solució: corregir l'error i tornar a engegar.

Problema: el sistema està lent

Diagnòstic:

```
htop
iostat -x 1
free -h
```

Causes comunes:

- Un procés consumint massa CPU.
- Swap ple (RAM insuficient).
- Disc al 100% I/O.
- Xarxa saturada.

Problema: "No space left on device"

Diagnòstic:

```
df -h
du -sh /var/lib/docker # contenidors
du -sh /var/log          # logs
du -sh /tmp              # temporals
```

Solució:

```
docker system prune -a # netejar Docker
find /var/log -name "*.gz" -delete
```

Problema: servei inaccessible des de la xarxa

Diagnòstic:

```
ss -tulnp | grep <port>
curl http://localhost:<port>
```

Si local funciona però remot no:

```
# Comprovar Tailscale
tailscale status
```

```
# Comprovar tallafocs
sudo ufw status
sudo iptables -L -n
```

```
# Comprovar ACLs a Tailscale
```

Problema: errors de permisos

Diagnòstic:

```
ls -la /path/fitxer
id
```

Solució:

```
sudo chown bernat:bernat /path/fitxer
sudo chmod 644 /path/fitxer
```

Problema: la Raspberry no respon

Diagnòstic:

1. Comprovar llum LED: vermell fix = problema d'alimentació.
2. Comprovar temperatura: si està molt calenta, ventilació.
3. Provar amb un altre cable d'alimentació.
4. Connectar monitor + teclat.

Si cap funciona, és problema de hardware (substituir).

56.4 Eines avançades

strace

Veure què fa exactament un procés:

```
strace -p <PID>
```

Útil quan un procés es queda penjat.

tcpdump

Capturar tràfic de xarxa:

```
sudo tcpdump -i any -n port 1883
```

Útil per veure si els missatges MQTT arriben.

iotop

Veure I/O de disc per procés:

```
sudo iotop
```

lsof

Veure fitxers oberts per un procés:

```
sudo lsof -p <PID>
```

nethogs

Veure tràfic de xarxa per procés:

```
sudo nethogs
```

56.5 Logs útils al BernatLab

- **/var/log/syslog**: sistema general.
- **/var/log/auth.log**: SSH, sudo.
- **/var/log/kern.log**: kernel.
- **/var/log/docker.log**: Docker daemon.
- **/var/log/fail2ban.log**: bloquejos.
- **/var/lib/docker/volumes/<volum>/_data/**: logs dins de contenidors.

56.6 El mètode de les 5 preguntes

Quan un problema sembla estrany, fes-te aquestes 5 preguntes:

1. **Què ha canviat recentment?** Sovint el problema ve d'un canvi.
2. **Què ha funcionat abans?** La configuració anterior és la base.
3. **Què és diferent ara?** Comparar l'estat anterior amb l'actual.
4. **Què passa si ho provo amb dades netes?** Sovint un estat net ho resol.
5. **Què passaria si fos un altre sistema?** Aïllar el problema.

56.7 Errors habituals en el diagnòstic

Error 1: canviar moltes coses alhora.

Si canvies 3 coses i es resol, no saps què ha funcionat. Canvia una cosa a la vegada.

Error 2: no documentar el que ja has provat.

Si proves 5 coses sense anotar, perds el temps repetint. Documenta cada intent.

Error 3: assumir la causa massa ràpid.

"És la memòria, segur". No, comprova-ho.

Error 4: Google com a primera opció.

Si no entens el problema, no busquis solucions. Enten-lo primer.

Error 5: ignorar els logs.

Els logs tenen la resposta. Llegeix-los abans de fer res.

56.8 Quan demanar ajuda

Si has provat tot i no trobes la solució, és hora de demanar ajuda. Però prepara't bé:

1. **Describeix el problema** amb detall.
2. **Mostra els logs** rellevants.
3. **Mostra què has provat.**
4. **Mostra la configuració** (sense secrets).
5. **Dona context:** quan va començar, què ha canviat.

Com pitjor estigui la descripció, pitjor serà l'ajuda que rebràs.

56.9 Post-mortem

Un cop resolt l'incident, escriu un **post-mortem** (un informe■■■■■):

- **Què ha passat:** descripció.
- **Quin impacte ha tingut:** temps de caiguda, dades perdudes.

- **Quina ha estat la causa arrel:** per què ha passat.
- **Què hem fet per resoldre:** com ho hem arreglat.
- **Què farem per prevenir:** quins canvis apliquem.

Exemple:

```
# Post-mortem: Tall de servei del 2026-07-05

## Què ha passat
El Grafana ha deixat de respondre durant 45 minuts entre les 14:00 i les 14:45.

## Impacte
- Usuaris no han pogut veure gràfiques.
- 0 dades perdudes (les dades estaven a InfluxDB, no a Grafana).

## Causa arrel
El contenidor Grafana ha omplert la partició /var. La causa: logs antics no rotats.

## Solució aplicada
- Netejar logs antics.
- Afegir logrotate.
- Configurar alerta quan /var > 80%.

## Prevenció
- Configurar `logrotate` per a Grafana.
- Moure els logs de Grafana a un volum separat.
- Afegir regla Prometheus: alerta si /var > 80%.
```

56.10 Documentació permanent

Un bon diagnòstic es converteix en:

- **Runbook** (Cap 55): si el problema es repeteix.
- **Post-mortem** (a [homelab/postmortems/](#)): si és greu.
- **Actualització del README**: si la solució canvia la configuració.
- **Alerta nova**: si vols prevenir-ho en el futur.

56.11 Resum

El diagnòstic és una habilitat que es millora amb la pràctica. Les 10 comandes bàsiques, el mètode general, els logs, i la calma són les eines. Documenta cada incident per aprendre'n. Al proper capítol veurem quan cal pujar de hardware i com planificar-ho.

56.12 Exercicis pràctics

1. Memoritza les 10 comandes bàsiques.
2. Prova cadascuna a la teva Raspberry.
3. Crea un "diccionari de problemes" amb els incidents que has tingut.
4. Escriu el teu primer post-mortem.
5. Configura `logrotate` per als serveis que ho necessitin.

6. Documenta al README les eines de diagnòstic instal·lades.

Capitol 57 — Quan cal pujar de hardware: escenaris reals

"Creixer és sa. Estancar-se no. Però créixer bé requereix planificació."

57.1 Senyals que cal pujar de hardware

Alguns senyals clars que la Raspberry ja no és prou:

- **CPU constantment al 80-90%** tot i no tenir pics.
- **Swap massa ple** sovint.
- **Disc ple** malgrat neteja regular.
- **Timeouts** a serveis web.
- **Contenidors que es reinicien** per manca de memòria.
- **Còpies lentes** que abans eren ràpides.
- **Múltiples usuaris concurrents** i el sistema no aguanta.

57.2 Quan és bona idea i quan no

Bona idea pujar:

- El sistema té **un paper real** (no és un experiment).
- La caiguda del servidor **té impacte** (pèrdua de dades, temps).
- Estàs **gastant més temps** mantenint el sistema que gaudint-lo.
- Has **assolit els límits** del hardware actual.

No és bona idea pujar:

- Per culpa d'**una mala configuració** que es pot millorar.
- Per **un sol servei** puntual que es pot optimitzar.
- Per **moda** o per tenir "el millor hardware".
- Sense **haver entès per què** el sistema actual no és prou.

57.3 Escenaris de pujada

Escenari 1: més RAM

Si la Raspberry té 4 GB i te'n calen 8, la solució és comprar una Raspberry amb 8 GB. Però:

- 8 GB és el màxim de la Raspberry 4.
- Si necessites més, cal canviar de plataforma.

Alternativa: **afegir swap a una SSD**:

```
# Crear swap a una SSD
sudo fallocate -l 8G /mnt/ssd/swapfile
sudo chmod 600 /mnt/ssd/swapfile
sudo mkswap /mnt/ssd/swapfile
sudo swapon /mnt/ssd/swapfile
```

Això et dona 8 GB extra de swap. No és tan ràpid com RAM, però ajuda.

Escenari 2: més emmagatzematge

Si la microSD de 64 GB no és prou:

1. Afegir una **SSD USB** de 256 GB o més.
2. Moure els volums Docker a la SSD.
3. Usar la microSD només per al sistema.

Exemple de configuració a `docker-compose.yml`:

```
volumes:
  grafana-data:
    driver_opts:
      type: none
      o: bind
      device: /mnt/ssd/grafana
```

Escenari 3: més potència de CPU

Si la CPU no és prou, les opcions són:

- **Raspberry Pi 5**: 2-3x més potent que la 4.
- **Mini PC amb Intel N100**: 4-8x més potent.
- **Servidor dedicat**: car però professional.

Escenari 4: alta disponibilitat

Si vols que el sistema no caigui mai:

- **Dues Raspberry** en failover (una activa, una standby).
- **Raspberry + cloud** (Raspberry activa, cloud com a backup).
- **Cluster de 3 Raspberry** amb replicació.

57.4 Comparació de plataformes

Plataforma	Preu	RAM	CPU	Consum	Per a què
Raspberry Pi 4 (4GB)	50 €	4 GB	4x ARM 1.8 GHz	5W	Aprenentatge, projectes petits
Raspberry Pi 4 (8GB)	75 €	8 GB	4x ARM 1.8 GHz	5W	BernatLab actual
Raspberry Pi 5 (8GB)	90 €	8 GB	4x ARM 2.4 GHz	8W	Més serveis, Més rapidesa
Mini PC N100	200-300 €	16 GB	4x x86 3.4 GHz	15-25W	Homelab seriós
Servidor dedicat	500+ €	32 GB+	Xeon / EPYC	100-300W	Empresa, virtualització
NAS Synology	300+ €	4-8 GB	ARM/x86	15-40W	Emmagatzematge + serveis
VPS al núvol	5-50 €/mes	1-16 GB	Variable	-	Sempre disponible, sense hardware

57.5 Migrar a una Raspberry Pi 5

Quan la 4 no és prou, la 5 és el pas natural:

Avantatges:

- CPU 2-3x més ràpida.
- Suport oficial per a SSD NVMe.
- Més ample de banda USB.
- Més memòria cau.

Inconvenients:

- Requereix font d'alimentació USB-C de 27W.
- Alguns contenidors poden tenir problemes amb la nova arquitectura.
- És més cara.

Procés de migració:

1. Comprar la Pi 5.
2. Flashear la microSD amb Raspberry Pi OS.
3. Instal·lar Docker, Tailscale, etc.
4. Restaurar la còpia de restic.
5. Reactivar els serveis.
6. Verificar que tot funciona.
7. Apagar la Pi 4 i posar-la com a backup.

57.6 Migrar a un Mini PC

Si la Raspberry ja no és prou, un mini PC és el pas següent:

Avantatges:

- Molta més potència.
- Suport per a molta RAM (fins a 64 GB).
- Suport per a múltiples discs.
- Compatibilitat amb programari x86 (més ampli).

Inconvenients:

- Més car.
- Consumeix més energia.
- Més soroll (ventiladors).

Models recomanats:

- Beelink Mini S12 (N100): 200 €, 16 GB RAM, 500 GB SSD.
- Intel NUC: 300-500 €.
- Lenovo ThinkCentre Tiny: 100-200 € (recondicionat).

57.7 Migrar al núvol

Per a alguns casos, el núvol és la millor opció:

Avantatges:

- Sense manteniment de hardware.
- Escalable.
- Accessible des de qualsevol lloc.
- Còpies automàtiques.

Inconvenients:

- Cost mensual.
- Dades fora de casa (privadesa).
- Depèn d'un proveïdor.

Quan té sentit:

- Quan vols alta disponibilitat.
- Quan no vols gestionar hardware.
- Quan el sistema és estrictament professional.

Proveïdors:

- Hetzner: 4 €/mes per un VPS bàsic.
- DigitalOcean: 6 \$/mes.
- AWS Lightsail: 5 \$/mes.

- Oracle Cloud: capa gratuïta generosa.

57.8 Estratègies híbrides

Pujar no és tot-o-res. Una estratègia híbrida:

- **Raspberry a casa:** serveis crítics, privats, dades locals.
- **VPS al núvol:** serveis públics, web, bot de Telegram.
- **Sincronització:** Tailscale uneix les dues parts.
- **Còpies:** el núvol rep còpies xifrades de la Raspberry.

Això et dona el millor dels dos mons.

57.9 Com planificar una migració

1. **Avaluar el sistema actual:** - Què funciona bé? - Què no funciona? - Quines són les limitacions?
1. **Definir els requisits:** - Quanta RAM necessito? - Quina CPU? - Quin emmagatzematge?
1. **Escollir la plataforma:** - Dins del pressupost? - Compleix els requisits? - Té el suport de la comunitat?
1. **Preparar la migració:** - Fer còpia completa. - Documentar l'estat actual. - Provar en un entorn separat.
1. **Migrar:** - En una finestra de manteniment. - Pas a pas. - Verificant cada pas.
1. **Verificar:** - Tots els serveis funcionen? - Les dades estan intactes? - El rendiment és l'esperat?
1. **Tancar l'antic sistema:** - Mantenir durant 1-2 setmanes com a backup. - Apagar quan tot està estable.

57.10 Cost total de propietat

No miris només el preu d'entrada. Considera:

- **Hardware:** preu del dispositiu.
- **Energia:** cost anual d'electricitat.
- **Manteniment:** temps dedicat.
- **Substitució:** cada 5-7 anys.
- **Còpies:** emmagatzematge extra al núvol.
- **Domini i DNS:** 10-15 €/any.

Exemple:

- Raspberry 4 4GB: $50 \text{ €} + 5\text{W} \times 24\text{h} \times 365 \times 0,15 \text{ €/kWh} = 6,57 \text{ €/any}$.
- Mini PC: $250 \text{ €} + 20\text{W} \times 24\text{h} \times 365 \times 0,15 \text{ €/kWh} = 26,28 \text{ €/any}$.
- VPS: 60 €/any.

A 5 anys, el mini PC costa 380 €, el VPS 300 €, la Raspberry 83 € + el teu temps.

57.11 Quan NO pujar

De vegades, la solució no és pujar de hardware:

- **Millor la configuració:** sovint es pot optimitzar.
- **Reduir serveis:** menys serveis = menys recursos.
- **Moure al núvol:** pot ser més barat.
- **Acceptar els límits:** si el sistema funciona per al que el necessites, està bé.

57.12 Hardware específic recomanat

Per al BernatLab actual i futur:

- **SSD USB 3.0 de 500 GB** (Kingston A400, Samsung T7): ~50 €.
- **MicroSD A2 de 64 GB** (SanDisk Extreme): ~15 €.
- **Font d'alimentació oficial** de Raspberry: ~10 €.
- **Carcassa amb ventilador:** ~10-20 €.
- **Cables Ethernet Cat 6:** ~5-10 €/u.

Si escales:

- **Raspberry Pi 5 8 GB:** ~90 €.
- **Mini PC Intel N100:** ~250-350 €.
- **SSD NVMe 1 TB:** ~80 €.

57.13 Cicle de vida del hardware

- **Raspberry Pi 4:** llançada 2019, suportada fins ~2030.
- **Raspberry Pi 5:** llançada 2023, suportada fins ~2034.
- **Mini PC:** depèn del model, 5-7 anys.
- **Servidors dedicats:** 5-10 anys amb manteniment.
- **VPS:** sense vida física, però el proveïdor pot tancar.

57.14 On posar el hardware

Si canvies a un mini PC:

- **Habitació amb temperatura estable** (no golfes ni garatge).
- **Ventilació adequada.**
- **Protecció contra pols.**

- **UPS** (Sistema d'alimentació ininterrompuda) per a talls de llum.
- **Protecció contra sobretensions** (regleta amb protecció).

Un mini PC pot ser al costat del router, a la sala d'estar, o al despatx. Silenciós i petit.

57.15 Resum

Pujar de hardware és una decisió que s'ha de prendre amb temps. Avalua els requisits, planifica la migració, i no tinguis por de mantenir la Raspberry més temps del que toca. Al proper mòdul veurem com implementar el domini propi i la web.

57.16 Exercicis pràctics

1. Monitora el sistema actual durant una setmana. Identifica colls d'ampolla.
2. Calcula el cost total de propietat de la teva Raspberry.
3. Escribe un document d'avaluació: "Quan puc necessitar pujar de hardware?"
4. Compara 2-3 opcions de hardware per al futur.
5. Documenta un pla de migració pas a pas.
6. Fes una llista de "què emportar" si canvies de plataforma.
7. Estableix una data límit per revisar el rendiment.